Runtime and Memory Scaling of Naive Dense Matrix Multiplication in Python, Java and C

Big Data – Individual Assignment 01

Daniel Rodríguez Alonso
Grado en Ciencia e Ingeniería de Datos
Universidad de Las Palmas de Gran Canaria (ULPGC)
Big Data, Academic Year 2026–2027

***Abstract*—This study compares the runtime and memory of the same naive triple-loop (ijk) float64 matrix multiplication implemented in C (gcc -O2), Java (OpenJDK 21) and Python (CPython 3.12) on one Intel Core i7-9750H laptop, for square matrices with n from 64 to 2048. The three programs read shared input files and produce bit-identical results. Only the kernel is timed, with five separate process runs per configuration (three for Python at $n = 1536$), summarised by the median and the interquartile range. C and Java completed all sizes. Python completed $n = 1536$ (median 280 s) and exceeded the 600 s limit at $n = 2048$. The fitted growth exponent is 3.11 [95 % CI 2.58, 3.65] for Python but 3.45 [3.05, 3.86] for C and 3.86 [3.45, 4.27] for Java, because their time per multiply–add rises with n (C: from 0.714 to 4.51 ns). Java is indistinguishable from C for $128 \leq n \leq 384$ and up to 2.48 times slower for larger n; Python is 28.2–59.5 times slower than C. At $n = 1536$ the peak resident memory above the runtime baseline is 270.9 MiB for Python against 54.0 MiB for C. The conclusions are limited to this machine, these toolchains and these settings.**

***Index Terms*—matrix multiplication, benchmarking, performance evaluation, programming languages, JIT compilation, memory hierarchy, peak resident set size**

I. Introduction and Objective

Dense matrix multiplication is a building block of many data-intensive workloads (linear models, similarity computations, neural-network layers). Its classical definition needs n^3 multiply–add operations for two $n \times n$ matrices, so the cost of an implementation grows quickly with the data size, and the constant factor in front of n^3 is decided by the language runtime, the data layout and the memory hierarchy. This report is the first of four related studies; it establishes a validated sequential baseline that the later assignments (optimisation, sparse matrices, parallel and distributed execution) reuse.

Research question (RQ). *On one Intel Core i7-9750H laptop, how do the runtime and the memory use of the same naive triple-loop float64 matrix multiplication grow with the matrix size n when it is implemented in C, Java and Python, and how large are the differences between the three implementations at equal n?* The factor varied is n (side length of square $n \times n$ matrices) for each of three implementations; the outcomes are the kernel runtime, the time per multiply–add, the peak memory of the process and the largest feasible n under fixed budgets.

Hypotheses (predictions stated before the measurements). *H1:* the runtime of every implementation grows approximately as n^3 (fitted exponent $b \approx 3$), with possible deviations at large n when the working set leaves the caches. *H2:* at equal n the order is C $<$ Java $<$ Python; after just-in-time (JIT) warm-up Java is within a small factor of C, while Python is one to two orders of magnitude slower. *H3:* Python needs several times more memory per matrix element than C and Java, because every element is a boxed object.

The study contributes (i) three implementations with identical loop order that produce bit-identical results, (ii) a protocol with shared input files, a common timing boundary and recorded failures, and (iii) runtime, growth-exponent and memory measurements with their variability.

II. Background and Related Work

Classical algorithm. For $C = AB$, $c_{ij} = \sum_{k=0}^{n-1} a_{ik} b_{kj}$, which costs n^3 multiplications and n^3 additions ($2n^3$ floating-point operations); the three nested loops can be written in six orders that perform the same arithmetic but access memory differently [1]. In the ijk order with row-major storage used here, the innermost loop reads a row of A with unit stride and a column of B with stride n, i.e. every access to B touches a different cache line. Once B ($8n^2$ bytes) no longer fits in a cache level, the column walk causes cache and translation-lookaside-buffer (TLB) misses; Drepper uses exactly this matrix multiplication to illustrate the effect of access patterns on the memory hierarchy [2], and Lam et al. show that cache interference in matrix kernels depends strongly on the access stride and the matrix size [3]. High-performance libraries avoid these costs with blocking and packing [4]; such optimisations are deliberately excluded here and are the subject of Assignment 2.

Language runtimes. C is compiled ahead of time to machine code. Java bytecode is first interpreted and then compiled at run time by the HotSpot JIT compiler, so early iterations are slower and timings vary between virtual-machine invocations; Georges et al. recommend steady-state measurements and several VM invocations for this reason [5]. Since JDK 17 all Java floating-point operations are strict [6]. CPython 3.12 executes bytecode in an interpreter that specialises instructions adaptively at run time but does not generate machine code [7]; each Python

`float` is a separate heap object, and a `list` stores references to such objects. `sys.getsizeof` reports only the memory directly attributed to an object, not to the objects it refers to [8].

Numerical accuracy. A dot product of length n computed in floating point satisfies $|\hat{c}_{ij} - c_{ij}| \leq \gamma_n(|A||B|)_{ij}$ with $\gamma_n = nu/(1-nu)$ and unit round-off $u = 2^{-53}$ for float64 [9]. Different summation orders therefore give slightly different results, while the same order and the same IEEE 754 operations give identical results.

Benchmarking methodology. Hoefler and Belli recommend reporting medians and non-parametric variability measures, the number of repetitions and the full experimental setup [10]; Mytkowicz et al. show that seemingly innocuous aspects of the setup can bias conclusions [11], which motivates scoping the conclusions to the tested environment.

III. Methodology

A. Implementations

The three programs (`src/c`, `src/java`, `src/python`) implement the same kernel:

```
for i in 0..n-1:
  for j in 0..n-1:
    s = 0.0
    for k in 0..n-1: s += A[i][k] * B[k][j]
    C[i][j] = s
```

with float64 values (`double` in C and Java, `float` in Python), the same ijk order and a scalar accumulator, so the floating-point evaluation order is identical. Each language uses its idiomatic storage, which is the main unavoidable difference: *C* stores each matrix in one contiguous row-major block (`malloc`/`calloc`, element (i, j) at `M[i*n+j]`); *Java* uses `double[][]`, an array of separately allocated row arrays, so `b[k][j]` first loads the reference to row k and then the element, with bounds checks managed by the JIT; *Python* uses a list of row lists of `float` objects, so every access dereferences a pointer to a boxed value and every multiply–add is executed by the interpreter and allocates a new `float`. Java and Python hoist the row references `A[i]` and `C[i]` out of the inner loops; C computes the flat index. No library routine is used in any kernel; NumPy [12] is used only to generate the inputs and as the reference in the correctness tests.

Fixed settings. C: gcc 13.3.0 with `-O2 -std=c11 -Wall -Wextra`, default x86-64 target (SSE2, no `-march`), no `-ffast-math`, single thread; the generated assembly of the kernel contains only scalar `mulsd`/`addsd` instructions (no vectorisation and no fused multiply–add, FMA). Java: OpenJDK 21.0.12.1, `javac -release 21`, `java -Xms1g -Xmx4g -XX:+UseG1GC`, default tiered JIT. Python: CPython 3.12.3, standard library only. The kernel runs on one thread in every language (the JVM keeps its background JIT and garbage-collection threads); `OMP_NUM_THREADS`, `OPENBLAS_NUM_THREADS` and `MKL_NUM_THREADS` are set to 1 for every run.

TABLE I
Execution environment (`results/environment.md`, captured by `capture_env.sh`).

Item	Value
CPU	Intel Core i7-9750H, 1 socket, 6 cores / 12 threads, 2.6 GHz base, 4.5 GHz max, AVX2/FMA
Caches	L1d 32 KiB and L2 256 KiB per core; L3 12 MiB shared
Frequency	governor `powersave`, turbo enabled, AC power connected
Memory	15 GiB (16.6 GB) RAM
OS	Ubuntu 24.04.5 LTS, Linux 7.0.0-38-generic, x86-64
C	gcc 13.3.0, `-O2 -std=c11 -Wall -Wextra`
Java	OpenJDK 21.0.12.1 (HotSpot 64-bit Server VM), `-Xms1g -Xmx4g -XX:+UseG1GC`
Python	CPython 3.12.3 (venv)
Tools	NumPy 2.5.4 (inputs, reference), GNU `time` (RSS)
Threads	kernel single-threaded; `OMP/OPENBLAS/MKL_NUM_THREADS=1`

B. Inputs

All programs read the same files, generated once by `scripts/gen_inputs.py`: NumPy `default_rng(42)` (PCG64), A drawn first and then B, independent uniform values in $[-1, 1)$, stored as raw little-endian float64 in row-major order with a JSON sidecar and SHA-256 checksums. Since $|c_{ij}| \leq n$, no overflow can occur. Loading the files, building the language-specific structure and allocating C happen before the timed region.

C. Experimental environment

Table I summarises the machine. The laptop was shared with three other benchmark sessions of related assignments; all timed runs were serialised by a machine-wide file lock (Section III-E).

D. Correctness validation

Before any benchmark, `tests/test_correctness.py` runs the three benchmarked programs as black boxes and checks (Table II): known products with exact answers (1×1, 2×2 and a non-symmetric 3×3 with small integers), $AI = A$, $IB = B$ and $A \cdot 0 = 0$ for $n = 7$ (exact), and random uniform inputs for $n \in \{7, 64, 100\}$ against NumPy with the element-wise tolerance $|c_{ij} - c_{ij}^{\text{ref}}| \leq 2nu(|A||B|)_{ij}$. The factor 2 accounts for both results carrying an error of at most $\gamma_n(|A||B|)_{ij} \approx nu(|A||B|)_{ij}$, since NumPy sums in a different order. Results of the three own implementations must be bit-identical, because they perform the same IEEE 754 operations in the same order. During the benchmark the checksum $\sum_{ij} c_{ij}$ printed after every repetition is compared across languages at every n.

E. Measurement protocol and budgets

Timer and boundary. Each program measures the kernel only (boundary *kernel*) with a monotonic clock: `clock_gettime(CLOCK_MONOTONIC)` in C, `System.nanoTime()` in Java and `time.perf_counter_ns()` in Python. Input loading, allocation and zeroing of C and the checksum are excluded.

Printing the checksum consumes the result, which prevents dead-code elimination. For short kernels a sample times a batch of $\lceil 0.2\,\mathrm{s}/t_\mathrm{w} \rceil$ consecutive calls (t_w: last warm-up time) and reports the time per multiplication; the batch size is stored with every row.

Repetitions and warm-up. A configuration (language, n) consists of P separate process runs; each process performs W in-process warm-up repetitions (recorded, excluded from statistics) and one measured repetition, giving $k = P$ samples that include between-process variability [5]. A pilot (`results/raw/pilot_size_sweep.csv`) showed a strong JIT effect in Java and no warm-up effect in C at $n = 1024$ or in Python at $n = 512$. Within the 90-minute budget this led to: C $W = 2$ ($n \le 512$), 1 ($n = 768$), 0 ($n \ge 1024$); Java $W = 5$ ($n \le 512$), 2 ($n \le 1024$), 1 ($n \ge 1536$); Python $W = 1$ ($n \le 512$), 0 ($n \ge 768$); $P = 5$ everywhere except Python at $n = 1536$ ($P = 3$, about six minutes per run) and $n = 2048$ (one attempt to document the time limit). The order of the languages was shuffled at every n (fixed seed).

Memory. The common metric is the *peak resident set size* (RSS) of the whole process, measured by GNU `time` (`%M`) [13] for every process run; it includes the interpreter or virtual machine. To separate this runtime baseline, the same programs were run at $n = 1$ ($k = 5$). Each program also reports one language-internal metric, which is not comparable across languages: C `matrix_storage_bytes` (the exact $3{\cdot}8n^2$ bytes allocated), Java `jvm_heap_peak_used_bytes` (sum of the peak usage of the heap `MemoryPoolMXBeans` [14]) and Python `pyobj_deep_size_bytes` (`sys.getsizeof` of the outer lists, row lists and every element, measured outside the timed region).

Sizes, budgets and stopping rule. Declared sequence $n \in \{64, 128, 256, 384, 512, 768, 1024, 1536, 2048\}$, mixing powers of two with $384 = 3 \cdot 2^7$, 768 and 1536. Budgets fixed before the large runs: 600 s per process run, 8 GB of memory, at most 90 minutes of benchmark time for the whole study. Dense storage needs $3 \cdot 8n^2$ bytes (96 MiB at $n = 2048$), so memory is not the limiting resource. A run that exceeds a limit is recorded with its status and an empty time, and larger sizes of that language are not attempted.

Execution and statistics. Every configuration ran through `run_measured.py`, which holds a machine-wide lock while measuring, records the process wall time, peak RSS, exit status and the 1-minute load average, and appends one raw CSV row per repetition. The sweep was interrupted and resumed, so it ran in 3 measurement sessions (a new session starts after a gap of more than one hour between rows): session 1, 2026-10-27 22:48–23:01, all languages for $n \le 512$ plus C, Java and three Python runs at $n = 768$ (median load average 2.64, maximum 8.84, with a browser and other assignments active); session 2, 2026-10-29 10:46–11:00 after a fresh boot, the remaining two Python runs at $n = 768$, all languages at $n = 1024$ and the first Python run at $n = 1536$ (median load 1.19); session 3, 2026-10-29 16:57–17:34, the rest of $n = 1536$, $n = 2048$ and the C flags study (median load 1.06). In total the sweep produced 364 raw rows (235 of them warm-ups) and used about 57 minutes of process time. For each configuration the report gives the median and the interquartile range (IQR $= Q_3 - Q_1$) of the k measured samples. No measurement was excluded. The growth exponent b in $T \approx c\,n^b$ is fitted by least squares on $\log T$ vs. $\log n$ of the medians, with a 95 % confidence interval from the Student t distribution; runtime ratios at equal n use the ratio of medians with a 95 % bootstrap interval (5000 resamples).

TABLE II
CORRECTNESS TESTS (ALL IMPLEMENTATIONS RUN THROUGH THE BENCHMARKED COMMAND-LINE PROGRAMS). "EXACT": BIT-WISE EQUAL TO THE EXPECTED MATRIX; ERROR VS. NUMPY: MAXIMUM ABSOLUTE ERROR AND MAXIMUM RATIO TO THE BOUND $2nu(|A||B|)_{ij}$ (PASS IF ≤ 1).

Case	n	Check (C, Java, Python)	Identical
known_1x1	1	exact	yes
known_2x2	2	exact	yes
known_3x3_nonsymmetric	3	exact	yes
identity_right_A_I	7	exact	yes
identity_left_I_B	7	exact	yes
zero_A_0	7	exact	yes
random_uniform_n7	7	max err 2.2e-16, ratio 0.14	yes
random_uniform_n64	64	max err 2.7e-15, ratio 0.01	yes
random_uniform_n100	100	max err 3.6e-15, ratio 0.01	yes

Separate experiment: C compiler flags. Outside the controlled comparison, the same C source was compiled with `-O0`, `-O2`, `-O3` and `-O3 -march=native` (all with `-std=c11`) [15] and run at $n \in \{256, 512, 1024\}$ with $P = 5$ process runs of $W = 1$ warm-up and one measured repetition each (`results/raw/c_flags.csv`).

IV. RESULTS

A. Correctness

All 9 test cases passed for the three implementations (Table II). The known, identity and zero cases are reproduced exactly; against NumPy the largest absolute error is 3.55×10^{-15} and the largest error is 0.141 times the tolerance bound. In every case the C, Java and Python results are bit-identical. During the benchmark, the checksums of all completed runs agree across languages at all 9 sizes (at $n = 2048$ only C and Java completed).

B. Runtime

Table III lists the median kernel runtime, the IQR, the number of samples k and the time per multiply–add for every configuration, and Figure 1 plots the medians. At the largest common size $n = 1536$ the medians are 9.9 s (C, IQR 457 ms), 24.6 s (Java, IQR 163 ms) and 280 s (Python, $k = 3$, IQR 7 s); at $n = 2048$, C needs 38.8 s and Java 64 s.

Time per operation. Figure 2 divides each runtime by the n^3 multiply–adds. For C it rises from 0.714 ns at $n = 64$ to 1.36 ns at $n = 256$, stays between about 1.3 and 1.9 ns up to $n = 1024$, and then increases to 2.73 ns ($n = 1536$) and 4.51 ns ($n = 2048$). Java stays close to C up to $n = 768$ (1.02–2.11 ns) and then rises more steeply: 4.16 ns at $n = 1024$, 6.78 ns at $n = 1536$ and 7.44 ns at $n = 2048$. Python needs 42.5 ns at

TABLE III

KERNEL RUNTIME PER MULTIPLICATION: MEDIAN AND INTERQUARTILE RANGE (IQR) OVER k MEASURED SAMPLES (WARM-UPS EXCLUDED), TIME PER MULTIPLY–ADD T/n^3, AND MEDIAN PEAK RESIDENT SET SIZE (RSS, GNU `TIME`) OF THE WHOLE PROCESS. TIMER BOUNDARY: KERNEL ONLY. "–": NOT RUN (SEE TABLE V).

	C (gcc -O2)					Java (OpenJDK 21)					Python (CPython 3.12)				
n	Median	IQR	k	ns/madd	RSS [MiB]	Median	IQR	k	ns/madd	RSS [MiB]	Median	IQR	k	ns/madd	RSS [MiB]
64	187 µs	7.59 µs	5	0.71	2.0	278 µs	16.5 µs	5	1.06	49.3	11.1 ms	136 µs	5	42.46	12.8
128	1.89 ms	76.1 µs	5	0.90	2.3	2.13 ms	89.4 µs	5	1.02	49.4	95.8 ms	10.5 ms	5	45.68	14.4
256	22.8 ms	226 µs	5	1.36	3.4	23.6 ms	666 µs	5	1.40	50.7	828 ms	88.5 ms	5	49.38	19.9
384	106 ms	24.5 ms	5	1.88	5.3	100 ms	21.8 ms	5	1.77	52.5	6.1 s	1.12 s	5	107.78	29.4
512	237 ms	8.74 ms	5	1.76	8.0	284 ms	36.5 ms	5	2.11	54.9	7.85 s	955 ms	5	58.51	42.5
768	597 ms	27.8 ms	5	1.32	15.5	764 ms	127 ms	5	1.69	62.7	33 s	4.36 s	5	72.79	79.9
1024	2.02 s	283 ms	5	1.88	25.8	4.47 s	415 ms	5	4.16	72.4	77.8 s	3.13 s	5	72.43	132.9
1536	9.9 s	457 ms	5	2.73	55.8	24.6 s	163 ms	5	6.78	142.4	280 s	7 s	3	77.13	283.4
2048	38.8 s	592 ms	5	4.51	97.9	64 s	1.62 s	5	7.44	184.7	timeout (> 600 s)				–

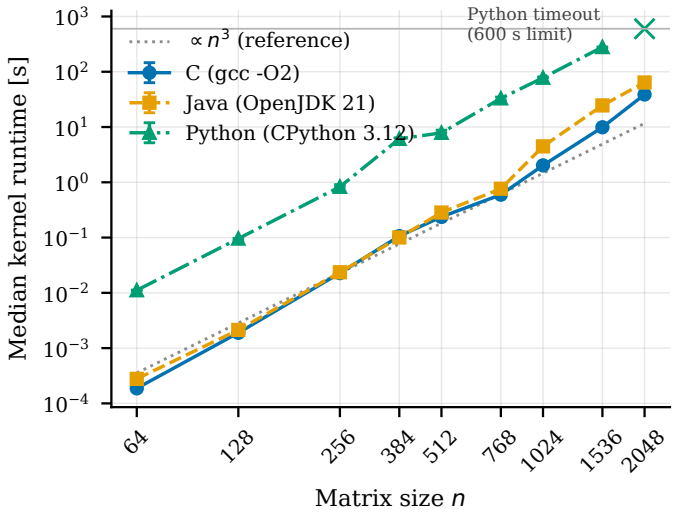


Fig. 1. Median kernel runtime per multiplication vs. matrix size (log–log; error bars: IQR, $k = 5$ except Python $n = 1536$, $k = 3$). Dotted line: n^3 reference anchored at the C median for $n = 256$. Cross: Python run stopped by the 600 s limit at $n = 2048$ (no time is reported for it).

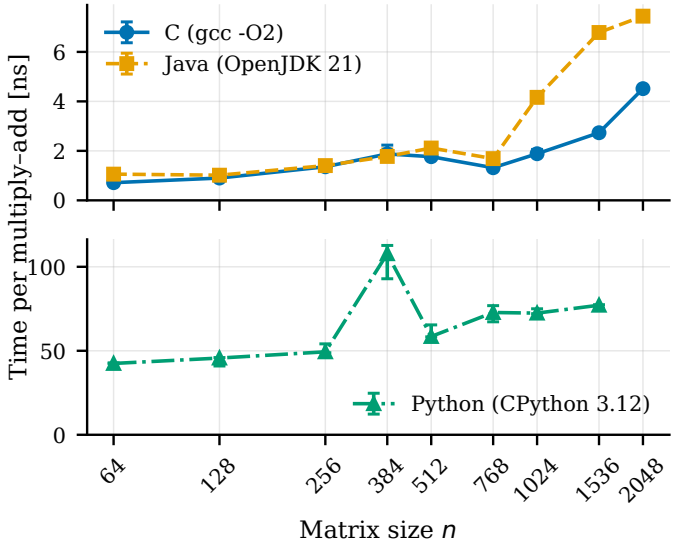


Fig. 2. Time per multiply–add, T/n^3 (median and IQR of the per-sample values). A flat curve means exact cubic growth. Python is shown on its own axis because its values are one to two orders of magnitude larger.

$n = 64$ and between 72.4 and 77.1 ns for $n \geq 768$, with an outlier of 108 ns at $n = 384$.

Ratios to C at equal n (ratio of medians, 95 % bootstrap interval). Java/C is 1.48 [1.3, 1.73] at $n = 64$; between $n = 128$ and $n = 512$ it lies between 0.946 and 1.2, and the intervals at $n = 128$, 256 and 384 include 1. For larger n it is 1.28 [1.08, 1.9] at $n = 768$, 2.21 [1.94, 3.03] at $n = 1024$, 2.48 [2.28, 2.69] at $n = 1536$ and 1.65 [1.62, 1.7] at $n = 2048$. Python/C is largest at $n = 64$, 59.5 [52.1, 62.2], and falls, not monotonically, to 38.4 [34.7, 47.5] at $n = 1024$ and 28.2 [25.9, 30.5] at $n = 1536$.

C. Growth exponent

Table IV gives the fitted exponents. Over $n = 256$–2048, C grows with $b = 3.45$ [3.05, 3.86] and Java with $b = 3.86$ [3.45, 4.27]; neither interval contains 3. Python grows with $b = 3.11$ [2.58, 3.65] over $n = 256$–1536, an interval that contains 3. The restricted fits show the same pattern: for C and Java the exponent is above 3 already for $n \leq 512$, and the large-n fits ($n \geq 512$) have wide intervals because they rest on only five or four points.

D. Limits

C and Java completed every declared size up to $n = 2048$ (Table V). The single Python attempt at $n = 2048$ was stopped by the 600 s limit (status `timeout`); it is recorded without a time. The largest feasible Python size under the declared budget is therefore $n = 1536$. No configuration came close to the 8 GB memory budget.

E. Memory

The runtime alone, measured at $n = 1$, occupies 1.9 MiB (C), 48.2 MiB (Java) and 12.5 MiB (Python) of peak RSS. For small n this baseline dominates (Figure 3): Java stays near 50 MiB up to $n = 512$. At the largest common size $n = 1536$ the peak RSS is 55.8 MiB (C), 142.4 MiB (Java) and 283.4 MiB (Python); after subtracting the baseline this is 54.0, 94.2 and 270.9 MiB, compared with 54.0 MiB for three dense float64

TABLE IV

EMPIRICAL GROWTH EXPONENT b OF $T \approx c\,n^b$ (LEAST SQUARES ON $\log$ MEDIAN RUNTIME VS. $\log n$) WITH 95 % CONFIDENCE INTERVAL (STUDENT t ON THE SLOPE). THE CUBIC MODEL PREDICTS $b = 3$.

Implementation	n range	pts	b [95 % CI]	R^2
C	256–2048	7	3.45 [3.05, 3.86]	0.9896
C	64–512	5	3.49 [3.31, 3.67]	0.9992
C	512–2048	5	3.73 [2.90, 4.57]	0.9854
Java	256–2048	7	3.86 [3.45, 4.27]	0.9915
Java	64–512	5	3.35 [3.10, 3.61]	0.9983
Java	512–2048	5	4.11 [3.19, 5.04]	0.9853
Python	256–1536	6	3.11 [2.58, 3.65]	0.9849
Python	64–512	5	3.29 [2.68, 3.91]	0.9899
Python	512–1536	4	3.23 [2.88, 3.59]	0.9987

TABLE V

FEASIBILITY LIMITS UNDER THE DECLARED BUDGETS (600 S PER PROCESS RUN, 8 GB MEMORY, $\leq$ 90 MIN TOTAL BENCHMARK TIME).

Impl.	Largest completed n	Largest attempted n	Outcome
C	2048	2048	none (end of sequence)
Java	2048	2048	none (end of sequence)
Python	1536	2048	timeout at $n = 2048$

matrices (Table VI). The language-specific metrics give 8.0 B per element for C (exact allocation), 11.5 B per element for the Java heap peak at $n = 2048$ and 32.0 B per element for the Python object graph.

F. Warm-up and variability

In Java the first in-process iteration takes 6.6 times the steady-state median at $n = 64$, while at $n = 2048$ it takes 1.01 times (Figure 4). In C and Python the first iteration is between 0.91–1.09 and 0.85–1.01 times the median, respectively, i.e. there is no systematic warm-up effect. The relative IQR (IQR/median) has a median over the sizes of 4.1 % (C), 5.9 % (Java) and 10.8 % (Python); its maximum occurs at $n = 384$ for all three languages (23.0 %, 21.7 % and 18.4 %), a size measured in session 1, when the machine was busy (load average up to 8.84 in that session). Two configurations mix sessions: Python at $n = 768$ (session 1 median 34.8 s, $k = 3$; session 2 30.4 s, $k = 2$) and Python at $n = 1536$ (session 2 268 s, $k = 1$; session 3 281 s, $k = 2$).

G. Separate experiment: C compiler flags

At $n = 1024$ (Table VII), `-O0` is 2.75 times slower than `-O2`, while `-O3` and `-O3 -march=native` take 0.957 and 0.964 times the `-O2` median. The checksums of the four variants are identical at every n. The `-O2` median of this experiment at $n = 1024$ (1.57 s, session 3) is lower than the 2.02 s measured for the same binary in the size sweep (session 2).

V. DISCUSSION

A. Answer to the research question

On the tested laptop, all three implementations perform exactly n^3 multiply–adds, but only the Python runtime grows

TABLE VI

MEMORY AT THE LARGEST COMPLETED n OF EACH IMPLEMENTATION. PEAK RSS: MEDIAN OVER THE MEASURED PROCESS RUNS (GNU `TIME` %M); BASELINE: MEDIAN PEAK RSS OF THE SAME PROGRAM AT $n = 1$ ($k = 5$). LANGUAGE METRIC: C `MATRIX_STORAGE_BYTES` (EXACT ALLOCATION), JAVA `JVM_HEAP_PEAK_USED_BYTES` (MEMORYPOOLMXBEAN, HEAP POOLS), PYTHON `PYOBJ_DEEP_SIZE_BYTES` (`SYS.GETSIZEOF`). THE METRICS MEASURE DIFFERENT THINGS AND ARE NOT DIRECTLY COMPARABLE.

Impl.	n	RSS [MiB]	Baseline [MiB]	RSS−base [MiB]	$3 \cdot 8n^2$ [MiB]	Lang. metric (B/elem.) [MiB]
C	1536	55.8	1.9	54.0	54.0	54.0 (8.0)
C	2048	97.9	1.9	96.0	96.0	96.0 (8.0)
Java	1536	142.4	48.2	94.2	54.0	96.9 (14.4)
Java	2048	184.7	48.2	136.5	96.0	138.4 (11.5)
Python	1536	283.4	12.5	270.9	54.0	216.3 (32.0)

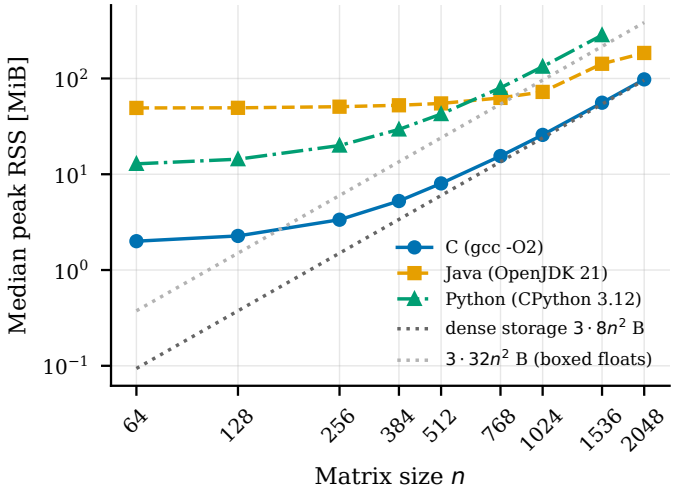


Fig. 3. Median peak RSS of the whole process (GNU `time`) vs. matrix size. Dotted lines: storage of the three matrices at 8 B per element (dense) and at 32 B per element (Python estimate: 24 B `float` object + 8 B list reference).

at a rate compatible with n^3 ($b = 3.11$). The C and Java runtimes grow faster ($b = 3.45$ and $b = 3.86$) because the time per multiply–add increases with n. At equal n, C and Java cannot be told apart for $128 \leq n \leq 384$; from $n = 768$ on, Java is slower by a factor between 1.28 and 2.48. Python is 28.2–59.5 times slower than C and is the only implementation that does not finish $n = 2048$ within 600 s. In memory, the fixed runtime cost dominates for small n (Java about 48.2 MiB), while for large n C uses exactly the dense storage and Python about five times as much.

B. Hypotheses

H1 (cubic growth) is partially supported. The operation count is cubic by construction and the Python exponent interval contains 3, but the C and Java intervals exclude 3 over the main range and for $n \leq 512$ (the fits for $n \geq 512$ have only five points and are too wide to decide for C). The predicted deviation at large n is present, and for C and Java it already appears at small n. *H2 (C < Java < Python, Java within a small factor) is supported with a qualification.* Python is

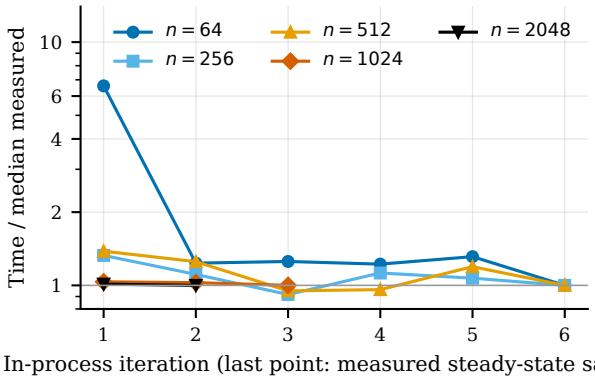


Fig. 4. Java JIT warm-up: time of each in-process warm-up iteration divided by the median of the measured samples at the same n (median over the $P = 5$ processes; log scale). The last point is the measured sample.

TABLE VII
SEPARATE EXPERIMENT (NOT PART OF THE LANGUAGE COMPARISON): SAME C SOURCE COMPILED WITH DIFFERENT GCC FLAGS. MEDIAN [IQR] KERNEL RUNTIME, $k = 5$ PROCESS RUNS, 1 WARM-UP EACH.

gcc flags	$n = 256$	$n = 512$	$n = 1024$
`-O0`	59.9 ms [2.27 ms]	495 ms [13 ms]	4.32 s [49 ms]
`-O2`	20.8 ms [1.64 ms]	180 ms [1.77 ms]	1.57 s [11 ms]
`-O3`	22.2 ms [268 µs]	176 ms [675 µs]	1.5 s [11.7 ms]
`-O3 -march=native`	20.5 ms [1.79 ms]	176 ms [4.5 ms]	1.51 s [23.8 ms]

always the slowest, and Java is at most 2.48 times slower than C. However, C is not measurably faster than Java at $n = 128$–384, so the order C $<$ Java holds only where the bootstrap intervals exclude 1 ($n = 64$ and $n \geq 512$). *H3 (more memory per element in Python) is supported.* `sys.getsizeof` gives 32.0 B per element against 8 B in C, and the peak RSS above the baseline at $n = 1536$ is 270.9 MiB against 54.0 MiB.

C. Interpretation

Memory hierarchy (C). The steps in the C time per multiply–add line up with the size of B ($8n^2$ B) relative to the caches of Table I. B fills the 32 KiB L1d at $n = 64$ (0.714 ns), fits in the 256 KiB L2 at $n = 128$ (0.901 ns), and needs the 12 MiB L3 at $n = 256$–1024 (8 MiB at $n = 1024$). It exceeds L3 at $n = 1536$ (18 MiB, 2.73 ns) and $n = 2048$ (32 MiB, 4.51 ns). In the ijk order every access to B jumps $8n$ bytes, so each one uses a new cache line and, for $n \geq 512$, a new 4 KiB page. This is the access pattern that Drepper uses to show the cost of cache and TLB misses [2]. The power-of-two sizes may add cache-set conflicts [3], which this design cannot separate from capacity effects. Hardware counters were not available (`perf_event_paranoid`$= 4$), so these explanations agree with the data but are *not verified* here.

Java layout and JIT. After warm-up, Java matches C while the working set is cache-resident, which indicates that the JIT-compiled loop is about as fast as the gcc `-O2` loop when memory is not the bottleneck. The gap opens exactly when C also slows down (from $n = 768$–1024), so Java appears to be more sensitive to the memory hierarchy. A plausible but unverified cause is the `double[][]` layout. Every access `b[k][j]` first loads the reference to row k, and the rows are separate heap objects with headers and no guaranteed contiguity or alignment. Confirming this would require cache-miss counters or the JIT-generated assembly. The JIT effect itself is clear in Figure 4. The first iteration is much slower only for short kernels, which run mostly in the interpreter or in early compilation tiers [5]. At $n \geq 1024$ the first iteration already takes about as long as the steady state, so JIT compilation happens within its first small fraction.

Interpreter cost (Python). CPython runs every multiply–add as several bytecode instructions with list indexing and creates a new `float` object for each product and partial sum [7], [8]. This fixed cost per operation, 42.5–77.1 ns compared with 0.714–4.51 ns in C, explains three observations. The exponent is close to 3, the ratio to C is large, and the ratio falls as n grows, because C becomes memory-bound while Python remains interpreter-bound. The moderate increase of the Python time per operation with n may also be a memory effect: the process reaches 283.4 MiB of peak RSS at $n = 1536$. This is a conjecture. The outlier at $n = 384$ was measured in the busiest session and is treated as measurement variability, not as a property of the algorithm.

Memory. For C the peak RSS above the baseline equals the dense storage (1.00× at $n = 2048$), as expected from three `malloc`ed blocks. The 32 B per element measured for Python matches a 24 B `float` object plus an 8 B list reference. The peak RSS is somewhat higher still (5.02× the dense storage, i.e. more than 4×). The remaining difference (allocator overhead and temporary buffers created while loading the inputs) is not attributed here. Java needs 1.42× the dense storage above its baseline at $n = 2048$. The JVM heap metric also includes objects that were not yet collected, so it is an upper bound of the live matrix data rather than an exact size.

Compiler flags. Turning optimisation off costs a factor 2.75, whereas `-O3` and `-march=native` change the time by less than 10 % at every tested n. A plausible explanation is the reduction `s += a*b`. GCC does not reassociate floating-point operations by default (`-fno-associative-math`), and no `-O` level below `-Ofast` enables this [15]. The compiler therefore has to keep the sequential order of the sum, which prevents the usual SIMD reduction. The identical checksums of all variants are consistent with an unchanged evaluation order.

D. Limitations

(i) One laptop with the `powersave` governor and turbo enabled. The clock frequency depends on temperature and on the number of active cores, and the three sessions ran under different loads. The same `-O2` binary gave 2.02 s at $n = 1024$ in session 2 and 1.57 s in session 3, so the difference between sessions can exceed the IQR within a session. Comparisons between sizes measured in different sessions should therefore be read with this uncertainty in mind. (ii) Other agents shared the machine. The lock serialised all timed runs, but not other work such as compilation. (iii) No hardware counters, so the explanations based on caches and

memory layout remain conjectures. (iv) Few samples where runs are expensive: $k = 3$ for Python at $n = 1536$ and a single attempt at $n = 2048$. Bootstrap intervals from $k = 5$ are approximate. (v) Each language uses its idiomatic layout. The results therefore compare implementations as written, not the languages in general. The language-specific memory metrics are not comparable with one another; only the peak RSS is. (vi) One input distribution (uniform $[-1, 1)$) and square matrices only.

VI. Conclusions

For the naive ijk float64 multiplication on an Intel Core i7-9750H laptop, with gcc 13.3 `-O2`, OpenJDK 21 and CPython 3.12, the study establishes the following points. (1) The three implementations are correct and bit-identical, so the comparison measures only how fast each one executes the same arithmetic. (2) The runtime grows faster than the cubic operation count for C and Java ($b \approx 3.45$ and 3.86), because the cost per multiply–add rises from 0.714 to 4.51 ns in C. For Python it grows roughly as n^3, because the interpreter cost per operation dominates. (3) Java is as fast as C for cache-resident sizes and up to 2.48 times slower for larger ones. Python is 28.2–59.5 times slower than C, and its largest feasible size under a 600 s budget is $n = 1536$. (4) The memory per element is 8 B in C and 32 B in Python, and the runtime baseline dominates the peak RSS of Java and Python at small n. These statements hold only for the tested implementations, settings, inputs and hardware. Several questions remain open. Hardware counters are needed to confirm the cache, TLB and layout explanations, including the specific cause of the Java gap. A contiguous one-dimensional Java array would show how much of the gap the layout explains. Measuring all sizes in one session would remove the variability between sessions. Loop reordering, blocking and parallel execution are left to the following assignments.

Reproducibility Materials

Repository: https://github.com/danirodriguezz/bigdata-assignment01-matmul-languages (submitted version: tag `v1.0-submission`); the same content is in `Assignment01_RodriguezAlonso_Materials.zip`. It contains the source code (`src/`), the correctness tests (`tests/`), the input generator and experiment driver (`scripts/`), the configuration (`configs/experiment.json`), all raw measurements (`results/raw/*.csv`, schema in `results/README.md`), the environment snapshot, the analysis scripts that produce every table, figure and number of this report (`analysis/make_all.py`) and this report's source. `make setup test inputs bench analysis report` reproduces the study; `make analysis report` regenerates all tables and figures from the committed raw data.

Use of Generative AI

Claude Code (Anthropic), model Claude Opus 5.5, was used as an assistant to plan the experiment, to draft and debug the C, Java and Python programs, the correctness tests, the benchmark driver and the analysis scripts, to draft the text of this report and the README, and for language revision. All measurements were produced by runs on the author's laptop and are stored in the repository; no result was generated or estimated by the tool. Every reference was checked against its DOI or publisher record or its official URL. The author reviewed the code, the data and the text and is responsible for the content.

References

[1] G. H. Golub and C. F. V. Loan, *Matrix Computations*, 4th ed. Baltimore, MD: Johns Hopkins University Press, 2013.

[2] U. Drepper, "What every programmer should know about memory," Red Hat, Inc., technical report, 2007, https://www.akkadia.org/drepper/cpumemory.pdf (accessed 2026-10-27).

[3] M. D. Lam, E. E. Rothberg, and M. E. Wolf, "The cache performance and optimizations of blocked algorithms," in *Proc. 4th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS-IV)*. ACM, 1991, pp. 63–74.

[4] K. Goto and R. A. van de Geijn, "Anatomy of high-performance matrix multiplication," *ACM Transactions on Mathematical Software*, vol. 34, no. 3, pp. 1–25, 2008.

[5] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*. ACM, 2007, pp. 57–76.

[6] OpenJDK, "JEP 306: Restore always-strict floating-point semantics," OpenJDK JDK Enhancement Proposal, delivered in JDK 17, https://openjdk.org/jeps/306 (accessed 2026-10-27).

[7] M. Shannon, "PEP 659 – specializing adaptive interpreter," Python Enhancement Proposals, 2021, https://peps.python.org/pep-0659/ (accessed 2026-10-27).

[8] Python Software Foundation, "sys — system-specific parameters and functions (`sys.getsizeof`), Python 3.12 documentation," https://docs.python.org/3.12/library/sys.html (accessed 2026-10-27).

[9] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics (SIAM), 2002.

[10] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC '15)*. ACM, 2015, pp. 1–12.

[11] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" in *Proc. 14th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS XIV)*. ACM, 2009, pp. 265–276.

[12] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[13] Linux man-pages project, "time(1) — run programs and summarize system resource usage," https://man7.org/linux/man-pages/man1/time.1.html (accessed 2026-10-27).

[14] Oracle, "Interface MemoryPoolMXBean, Java SE 21 & JDK 21 API specification," https://docs.oracle.com/en/java/javase/21/docs/api/java.management/java/lang/management/MemoryPoolMXBean.html (accessed 2026-10-27).

[15] Free Software Foundation, "Options that control optimization, GCC 13.3.0 manual," https://gcc.gnu.org/onlinedocs/gcc-13.3.0/gcc/Optimize-Options.html (accessed 2026-10-27).